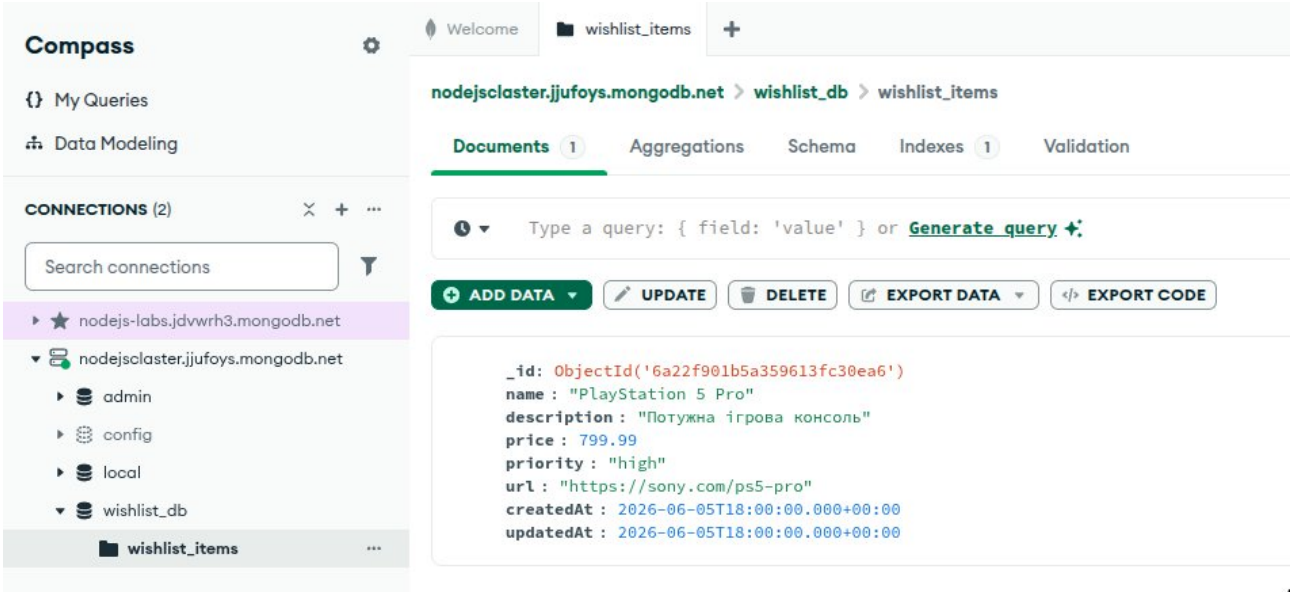
Лабораторна робота №4

Тема: MongoDB та Mongoose: міграція REST API на базу даних

Мета: набуття практичних навичок роботи з MongoDB та Mongoose шляхом міграції REST API з лабораторної роботи №3 з in-memory сховища на повноцінну базу даних, зокрема: • налаштування MongoDB Atlas та MongoDB Compass для хмарного зберігання та візуального управління даними; • підключення Mongoose до Express-застосунку з паттерном graceful shutdown; • проєктування Mongoose-схеми з валідацією, дефолтами та віртуальними властивостями; • міграція CRUD-операцій з Map-сховища на Mongoose Model; • централізована обробка специфічних помилок MongoDB (CastError, ValidationError, duplicate key); • реалізація фільтрації, сортування та пагінації через Mongoose Query API; • тестування API з використанням mongodb-memory-server замість реальної бази даних.

Хід роботи:

3.1. Завдання 1: MongoDB Atlas та MongoDB Compass



.env.example

```
MONGODB_URI=mongodb+srv://<username>:<password>@<cluster-url>/<database-name>
PORT=3000
```

3.2. Завдання 2: Підключення Mongoose та оновлення server.ts


```
{
 "name": "nodejs",
 "version": "1.0.0",
 "description": "",
 "main": "index.js",
 "scripts": {
  "dev": "ts-node src/server.ts",
  "build": "tsc",
  "test": "jest",
  "test:coverage": "jest --coverage"
 },
 "repository": {
  "type": "git",
  "url": "git+https://github.com/IhnatiukAndrii/nodejs-labs.git"
 },
 "keywords": [],
 "author": "",
 "license": "ISC",
 "type": "commonjs",
 "bugs": {
  "url": "https://github.com/IhnatiukAndrii/nodejs-labs/issues"
 },
 "homepage": "https://github.com/IhnatiukAndrii/nodejs-labs#readme",
 "dependencies": {
  "cors": "^2.8.6",
  "dotenv": "^17.4.2",
  "express": "^5.2.1",
  "mongoose": "^9.6.3",
  "zod": "^4.4.3"
 },
 "devDependencies": {
  "@types/cors": "^2.8.19",
```

```
    "@types/express": "^5.0.6",
    "@types/jest": "^30.0.0",
    "@types/supertest": "^7.2.0",
    "jest": "^30.4.2",
    "supertest": "^7.2.2",
    "ts-jest": "^29.4.11",
    "ts-node": "^10.9.2",
    "typescript": "^6.0.3"
  }
}
```

src/config/database.ts

```
import mongoose from 'mongoose';

export const connectDB = async (): Promise<void> => {
    const uri = process.env.MONGODB_URI;
    if (!uri) {
        throw new Error('MONGODB_URI is not defined in the environment variables');
    }

    mongoose.connection.on('error', (err) => {
        console.error(`Mongoose connection error: ${err}`);
    });

    mongoose.connection.on('disconnected', () => {
        console.log('Mongoose connection disconnected');
    });

    await mongoose.connect(uri);
    console.log('Successfully connected to MongoDB');
};
```

src/server.ts

```
import dotenv from 'dotenv';
dotenv.config();

import app from './app';
import { connectDB } from './config/database';
import mongoose from 'mongoose';

const PORT = process.env.PORT || 3000;

const startServer = async () => {
  try {
    await connectDB();

    const server = app.listen(PORT, () => {
      console.log(`Server is running on port ${PORT}`);
    });

    const shutdown = async () => {
      console.log('Shutting down server gracefully...');
      server.close(async () => {
        console.log('HTTP server closed.');
        await mongoose.connection.close();
        console.log('MongoDB connection closed.');
        process.exit(0);
      });
    };

    process.on('SIGTERM', shutdown);
    process.on('SIGINT', shutdown);
  } catch (error) {
    console.error('Failed to start server:', error);
    process.exit(1);
```

```
  }
};

startServer();
```

```
> nodejs@1.0.0 dev
> ts-node src/server.ts

◇ injected env (2) from .env // tip: ⌘ override existing { override: true }
Successfully connected to MongoDB
Server is running on port 3000
```

3.3. Завдання 3: Mongoose-схема та модель

```
import mongoose, { Schema } from 'mongoose';

export interface IWishlistItem {
  name: string;
  description?: string;
  price: number;
  priority: 'low' | 'medium' | 'high';
  url?: string;
}

const wishlistSchema = new Schema<IWishlistItem>(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      minlength: [1, 'Name must be at least 1 character long'],
      maxlength: [100, 'Name cannot exceed 100 characters'],
      trim: true
    },
```

```
    description: {
      type: String,
      maxlength: [500, 'Description cannot exceed 500 characters'],
      trim: true
    },
    price: {
      type: Number,
      required: [true, 'Price is required'],
      min: [0.01, 'Price must be greater than 0']
    },
    priority: {
      type: String,
      required: [true, 'Priority is required'],
      enum: {
        values: ['low', 'medium', 'high'],
        message: '{VALUE} is not a valid priority'
      },
      default: 'medium'
    },
    url: {
      type: String,
      validate: {
        validator: function (v: string) {
          if (!v) return true;
          try {
            new URL(v);
            return true;
          } catch (e) {
            return false;
          }
        },
        message: (props: any) => `${props.value} is not a valid URL!`
      }
```

```
    }
  },
  {
    timestamps: true,
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  }
);

wishlistSchema.virtual('isExpensive').get(function (this: any) {
  return this.price > 100;
});

export const WishlistItem = mongoose.model<IWishlistItem>('WishlistItem', wishlistSchema);
```

3.4. Завдання 4: Заміна сховища на Mongoose

src/storage/entity.ts

```
import { WishlistItem } from '../models/entity.model';
import { Entity, EntityFilters } from '../schemas/entity.schema';

export const entityStorage = {
 async findAll(filters?: EntityFilters): Promise<any[]> {
  const query: any = {};
  if (filters) {
   if (filters.priority) {
    query.priority = filters.priority;
   }
   if (filters.maxPrice !== undefined) {
    query.price = { $lte: filters.maxPrice };
   }
   if (filters.name) {
```

```
      query.name = { $regex: filters.name, $options: 'i' };
    }
  }
  return WishlistItem.find(query);
 },

 async findById(id: string): Promise<any | null> {
  return WishlistItem.findById(id);
 },

 async create(item: Omit<Entity, 'id' | 'createdAt' | 'updatedAt'>): Promise<any> {
  return WishlistItem.create(item);
 },

 async update(id: string, item: Partial<Omit<Entity, 'id' | 'createdAt' | 'updatedAt'>>):
Promise<any | null> {
  return WishlistItem.findByIdAndUpdate(id, item, {
   new: true,
   runValidators: true
  });
 },

 async delete(id: string): Promise<boolean> {
  const result = await WishlistItem.findByIdAndDelete(id);
  return result !== null;
 }
};
```

src/routes/entity.ts

```
import { Router } from 'express';
import { entityStorage } from '../storage/entity';
import { createSchema, updateSchema, EntityFilters } from '../schemas/entity.schema';
```

```
import { validate } from '../middleware';


const router = Router();


router.get('/', async (req, res, next) => {
  try {
    const { priority, maxPrice, name } = req.query;
    const filters: EntityFilters = {};
    if (typeof priority === 'string' && ['low', 'medium', 'high'].includes(priority)) {
      filters.priority = priority as 'low' | 'medium' | 'high';
    }
    if (typeof maxPrice === 'string' && !isNaN(Number(maxPrice))) {
      filters.maxPrice = Number(maxPrice);
    }
    if (typeof name === 'string') {
      filters.name = name;
    }
    const items = await entityStorage.findAll(filters);
    res.status(200).json(items);
  } catch (error) {
    next(error);
  }
});


router.get('/:id', async (req, res, next) => {
  try {
    const item = await entityStorage.findById(req.params.id as string);
    if (!item) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    res.status(200).json(item);
  } catch (error) {
```

```
    next(error);
  }
});

router.post('/', validate(createSchema), async (req, res, next) => {
  try {
    const newItem = await entityStorage.create(req.body);
    res.status(201).json(newItem);
  } catch (error) {
    next(error);
  }
});

router.put('/:id', validate(updateSchema), async (req, res, next) => {
  try {
    const updated = await entityStorage.update(req.params.id as string, req.body);
    if (!updated) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    res.status(200).json(updated);
  } catch (error) {
    next(error);
  }
});

router.delete('/:id', async (req, res, next) => {
  try {
    const success = await entityStorage.delete(req.params.id as string);
    if (!success) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
```

```
        res.status(204).end();
    } catch (error) {
        next(error);
    }
});

export default router;
```

tests/entity.test.ts

```
import request from 'supertest';
import app from '../src/app';
import { entityStorage } from '../src/storage/entity';
import { WishlistItem } from '../src/models/entity.model';
import { connectDB } from '../src/config/database';
import mongoose from 'mongoose';
import dotenv from 'dotenv';

// Load environment variables for testing
dotenv.config();

beforeAll(async () => {
    await connectDB();
});

afterAll(async () => {
    await mongoose.connection.close();
});

describe('Wishlist API Integration Tests', () => {
    beforeEach(async () => {
        await WishlistItem.deleteMany({});
    });
```

```
const validItem = {
    name: 'PlayStation 5',
    description: 'Next-gen gaming console',
    price: 499.99,
    priority: 'high' as const,
    url: 'https://sony.com/ps5'
};

it('GET /entities should return an empty array initially', async () => {
    const res = await request(app).get('/entities');
    expect(res.status).toBe(200);
    expect(res.body).toEqual([]);
});

it('GET /entities/:id should return 404 for non-existent ID', async () => {
    const res = await request(app).get('/entities/non-existent-uuid');
    expect(res.status).toBe(404);
    expect(res.body).toEqual({ error: 'Item not found' });
});

it('POST /entities should create a wishlist item and return 201', async () => {
    const res = await request(app).post('/entities').send(validItem);
    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty('id');
    expect(res.body).toHaveProperty('createdAt');
    expect(res.body).toHaveProperty('updatedAt');
    expect(res.body.name).toBe(validItem.name);
    expect(res.body.description).toBe(validItem.description);
    expect(res.body.price).toBe(validItem.price);
    expect(res.body.priority).toBe(validItem.priority);
    expect(res.body.url).toBe(validItem.url);
});
```

```
it('POST /entities should return 400 if name is empty', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, name: '' });
    expect(res.status).toBe(400);
    expect(res.body.status).toBe('error');
    expect(res.body.message).toBe('Validation error');
});

it('POST /entities should return 400 if name exceeds 100 characters', async () => {
    const longName = 'a'.repeat(101);
    const res = await request(app).post('/entities').send({ ...validItem, name: longName });
    expect(res.status).toBe(400);
    expect(res.body.status).toBe('error');
});

it('POST /entities should return 400 if description exceeds 500 characters', async () => {
    const longDesc = 'a'.repeat(501);
    const res = await request(app).post('/entities').send({ ...validItem, description: longDesc });
    expect(res.status).toBe(400);
});

it('POST /entities should return 400 if price is negative', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, price: -10 });
    expect(res.status).toBe(400);
});

it('POST /entities should return 400 if priority is invalid', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, priority: 'critical' });
    expect(res.status).toBe(400);
});

it('POST /entities should return 400 if url is invalid URL format', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, url: 'not-a-valid-url' });
```

```
    expect(res.status).toBe(400);
  });

  it('GET /entities/:id should return the item if it exists', async () => {
    const created = await entityStorage.create(validItem);
    const res = await request(app).get(`/entities/${created.id}`);
    expect(res.status).toBe(200);
    expect(res.body.id).toBe(created.id);
    expect(res.body.name).toBe(validItem.name);
  });

  it('PUT /entities/:id should update wishlist item and return 200', async () => {
    const created = await entityStorage.create(validItem);
    const updateData = { name: 'PlayStation 5 Slim', price: 449.99 };
    const res = await request(app).put(`/entities/${created.id}`).send(updateData);
    expect(res.status).toBe(200);
    expect(res.body.id).toBe(created.id);
    expect(res.body.name).toBe(updateData.name);
    expect(res.body.price).toBe(updateData.price);
    expect(res.body.priority).toBe(validItem.priority);
  });

  it('PUT /entities/:id should return 404 if updating non-existent item', async () => {
    const res = await request(app).put('/entities/non-existent-uuid').send({ name: 'New
Name' });
    expect(res.status).toBe(404);
  });

  it('DELETE /entities/:id should delete item and return 204', async () => {
    const created = await entityStorage.create(validItem);
    const delRes = await request(app).delete(`/entities/${created.id}`);
    expect(delRes.status).toBe(204);

    const getRes = await request(app).get(`/entities/${created.id}`);
```

```
    expect(getRes.status).toBe(404);
  });

  it('DELETE /entities/:id should return 404 if deleting non-existent item', async () => {
    const res = await request(app).delete('/entities/non-existent-uuid');
    expect(res.status).toBe(404);
  });

  it('GET /entities?priority=high should return only high-priority items', async () => {
    await entityStorage.create({ ...validItem, name: 'Item 1', priority: 'high' });
    await entityStorage.create({ ...validItem, name: 'Item 2', priority: 'low' });
    await entityStorage.create({ ...validItem, name: 'Item 3', priority: 'high' });

    const res = await request(app).get('/entities?priority=high');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(2);
    expect(res.body.every((item: any) => item.priority === 'high')).toBe(true);
  });

  it('GET /entities?maxPrice=100 should return only items with price <= 100', async () => {
    await entityStorage.create({ ...validItem, name: 'Cheap 1', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Expensive', price: 150 });
    await entityStorage.create({ ...validItem, name: 'Cheap 2', price: 100 });

    const res = await request(app).get('/entities?maxPrice=100');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(2);
    expect(res.body.every((item: any) => item.price <= 100)).toBe(true);
  });

  it('GET /entities?name=Play should return only items with matching names (case-
insensitive)', async () => {
    await entityStorage.create({ ...validItem, name: 'PlayStation 5' });
    await entityStorage.create({ ...validItem, name: 'Xbox Series X' });
```

```
    await entityStorage.create({ ...validItem, name: 'Nintendo Switch' });

    const res = await request(app).get('/entities?name=play');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(1);
    expect(res.body[0].name).toBe('PlayStation 5');
  });

  it('GET /entities with multiple filters should return only matching items', async () => {
    await entityStorage.create({ ...validItem, name: 'Low Cheap', priority: 'low', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Low Expensive', priority: 'low', price: 150 });
    await entityStorage.create({ ...validItem, name: 'High Cheap', priority: 'high', price: 50 });

    const res = await request(app).get('/entities?priority=low&maxPrice=100');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(1);
    expect(res.body[0].name).toBe('Low Cheap');
  });
});
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	91.53	84.21	90.9	91.86	
src	100	100	100	100	
app.ts	100	100	100	100	
src/config	83.33	50	66.66	81.81	
database.ts	83.33	50	66.66	81.81	6,10
src/middleware	95.23	25	100	93.75	
errorHandler.ts	85.71	25	100	83.33	19
index.ts	100	100	100	100	
validation.ts	100	100	100	100	
src/models	75	50	66.66	81.81	
entity.model.ts	75	50	66.66	81.81	48-51
src/routes	89.58	100	100	89.58	
entity.ts	89.58	100	100	89.58	24,37,46,59,72
src/schemas	100	100	100	100	
entity.schema.ts	100	100	100	100	
src/storage	100	92.85	100	100	
entity.ts	100	92.85	100	100	8

```
Test Suites: 1 passed, 1 total
Tests:       18 passed, 18 total
Snapshots:   0 total
Time:        11.524 s
```

3.5. Завдання 5: Обробка помилок MongoDB

src/middleware/errorHandler.ts

```
import { Request, Response, NextFunction } from 'express';
import { ZodError } from 'zod';

export const errorHandler = (
  err: any,
  req: Request,
  res: Response,
  next: NextFunction
): void => {
  if (err instanceof ZodError) {
    res.status(400).json({
      status: 'error',
      message: 'Validation error',
      errors: err.issues
    });
    return;
  }

  if (err.name === 'CastError') {
    res.status(400).json({
      status: 'error',
      message: 'Invalid ID format',
      details: err.message
    });
    return;
  }

  if (err.name === 'ValidationError') {
    res.status(400).json({
      status: 'error',
      message: 'Validation error',
```

```
            errors: Object.values(err.errors || {}).map((e: any) => ({
                path: e.path,
                message: e.message
            }))
        });
        return;
    }

    if (err.code === 11000 || err.code === 11001) {
        res.status(400).json({
            status: 'error',
            message: 'Duplicate key error',
            keyValue: err.keyValue
        });
        return;
    }

    res.status(500).json({
        status: 'error',
        message: err.message || 'Internal Server Error'
    });
};
```

tests/entity.test.ts

```
import request from 'supertest';
import app from '../src/app';
import { entityStorage } from '../src/storage/entity';
import { WishlistItem } from '../src/models/entity.model';
import { connectDB } from '../src/config/database';
import mongoose from 'mongoose';
import dotenv from 'dotenv';
```

```
// Load environment variables for testing
dotenv.config();

beforeAll(async () => {
  await connectDB();
});

afterAll(async () => {
  await mongoose.connection.close();
});

describe('Wishlist API Integration Tests', () => {
  beforeEach(async () => {
    await WishlistItem.deleteMany({});
  });

  const validItem = {
    name: 'PlayStation 5',
    description: 'Next-gen gaming console',
    price: 499.99,
    priority: 'high' as const,
    url: 'https://sony.com/ps5'
  };

  it('GET /entities should return an empty array initially', async () => {
    const res = await request(app).get('/entities');
    expect(res.status).toBe(200);
    expect(res.body).toEqual([]);
  });

  it('GET /entities/:id should return 400 for invalid ID format', async () => {
    const res = await request(app).get('/entities/non-existent-uuid');
    expect(res.status).toBe(400);
```

```
});

it('GET /entities/:id should return 404 for valid format but non-existent ID', async () => {
    const res = await request(app).get('/entities/507f1f77bcf86cd799439011');
    expect(res.status).toBe(404);
    expect(res.body).toEqual({ error: 'Item not found' });
});

it('POST /entities should create a wishlist item and return 201', async () => {
    const res = await request(app).post('/entities').send(validItem);
    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty('id');
    expect(res.body).toHaveProperty('createdAt');
    expect(res.body).toHaveProperty('updatedAt');
    expect(res.body.name).toBe(validItem.name);
    expect(res.body.description).toBe(validItem.description);
    expect(res.body.price).toBe(validItem.price);
    expect(res.body.priority).toBe(validItem.priority);
    expect(res.body.url).toBe(validItem.url);
});

it('POST /entities should return 400 if name is empty', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, name: '' });
    expect(res.status).toBe(400);
    expect(res.body.status).toBe('error');
    expect(res.body.message).toBe('Validation error');
});

it('POST /entities should return 400 if name exceeds 100 characters', async () => {
    const longName = 'a'.repeat(101);
    const res = await request(app).post('/entities').send({ ...validItem, name: longName });
    expect(res.status).toBe(400);
    expect(res.body.status).toBe('error');
```

```
});

it('POST /entities should return 400 if description exceeds 500 characters', async () => {
  const longDesc = 'a'.repeat(501);
  const res = await request(app).post('/entities').send({ ...validItem, description: longDesc });
  expect(res.status).toBe(400);
});

it('POST /entities should return 400 if price is negative', async () => {
  const res = await request(app).post('/entities').send({ ...validItem, price: -10 });
  expect(res.status).toBe(400);
});

it('POST /entities should return 400 if priority is invalid', async () => {
  const res = await request(app).post('/entities').send({ ...validItem, priority: 'critical' });
  expect(res.status).toBe(400);
});

it('POST /entities should return 400 if url is invalid URL format', async () => {
  const res = await request(app).post('/entities').send({ ...validItem, url: 'not-a-valid-url' });
  expect(res.status).toBe(400);
});

it('GET /entities/:id should return the item if it exists', async () => {
  const created = await entityStorage.create(validItem);
  const res = await request(app).get(`/entities/${created.id}`);
  expect(res.status).toBe(200);
  expect(res.body.id).toBe(created.id);
  expect(res.body.name).toBe(validItem.name);
});

it('PUT /entities/:id should update wishlist item and return 200', async () => {
  const created = await entityStorage.create(validItem);
```

```
    const updateData = { name: 'PlayStation 5 Slim', price: 449.99 };
    const res = await request(app).put(`/entities/${created.id}`).send(updateData);
    expect(res.status).toBe(200);
    expect(res.body.id).toBe(created.id);
    expect(res.body.name).toBe(updateData.name);
    expect(res.body.price).toBe(updateData.price);
    expect(res.body.priority).toBe(validItem.priority);
  });

  it('PUT /entities/:id should return 400 if updating invalid ID format', async () => {
    const res = await request(app).put('/entities/non-existent-uuid').send({ name: 'New Name' });
    expect(res.status).toBe(400);
  });

  it('PUT /entities/:id should return 404 if updating valid format but non-existent ID', async () => {
    const res = await request(app).put('/entities/507f1f77bcf86cd799439011').send({ name: 'New Name' });
    expect(res.status).toBe(404);
  });

  it('DELETE /entities/:id should delete item and return 204', async () => {
    const created = await entityStorage.create(validItem);
    const delRes = await request(app).delete(`/entities/${created.id}`);
    expect(delRes.status).toBe(204);

    const getRes = await request(app).get(`/entities/${created.id}`);
    expect(getRes.status).toBe(404);
  });

  it('DELETE /entities/:id should return 400 if deleting invalid ID format', async () => {
    const res = await request(app).delete('/entities/non-existent-uuid');
    expect(res.status).toBe(400);
```

```
  });

  it('DELETE /entities/:id should return 404 if deleting valid format but non-existent ID', async () => {
    const res = await request(app).delete('/entities/507f1f77bcf86cd799439011');
    expect(res.status).toBe(404);
  });

  it('GET /entities?priority=high should return only high-priority items', async () => {
    await entityStorage.create({ ...validItem, name: 'Item 1', priority: 'high' });
    await entityStorage.create({ ...validItem, name: 'Item 2', priority: 'low' });
    await entityStorage.create({ ...validItem, name: 'Item 3', priority: 'high' });

    const res = await request(app).get('/entities?priority=high');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(2);
    expect(res.body.every((item: any) => item.priority === 'high')).toBe(true);
  });

  it('GET /entities?maxPrice=100 should return only items with price <= 100', async () => {
    await entityStorage.create({ ...validItem, name: 'Cheap 1', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Expensive', price: 150 });
    await entityStorage.create({ ...validItem, name: 'Cheap 2', price: 100 });

    const res = await request(app).get('/entities?maxPrice=100');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(2);
    expect(res.body.every((item: any) => item.price <= 100)).toBe(true);
  });

  it('GET /entities?name=Play should return only items with matching names (case-insensitive)', async () => {
    await entityStorage.create({ ...validItem, name: 'PlayStation 5' });
    await entityStorage.create({ ...validItem, name: 'Xbox Series X' });
```

```
    await entityStorage.create({ ...validItem, name: 'Nintendo Switch' });

    const res = await request(app).get('/entities?name=play');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(1);
    expect(res.body[0].name).toBe('PlayStation 5');
  });

  it('GET /entities with multiple filters should return only matching items', async () => {
    await entityStorage.create({ ...validItem, name: 'Low Cheap', priority: 'low', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Low Expensive', priority: 'low', price: 150 });
    await entityStorage.create({ ...validItem, name: 'High Cheap', priority: 'high', price: 50 });

    const res = await request(app).get('/entities?priority=low&maxPrice=100');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(1);
    expect(res.body[0].name).toBe('Low Cheap');
  });
});
```

3.6. Завдання 6: Фільтрація, сортування та пагінація

src/routes/entity.ts

```
import { Router } from 'express';
import { entityStorage } from '../storage/entity';
import { createSchema, updateSchema, EntityFilters } from '../schemas/entity.schema';
import { validate } from '../middleware';

const router = Router();

router.get('/', async (req, res, next) => {
  try {
```

```
    const { priority, maxPrice, name, sort, page, limit } = req.query;
    const filters: EntityFilters = {};
    if (typeof priority === 'string' && ['low', 'medium', 'high'].includes(priority)) {
      filters.priority = priority as 'low' | 'medium' | 'high';
    }
    if (typeof maxPrice === 'string' && !isNaN(Number(maxPrice))) {
      filters.maxPrice = Number(maxPrice);
    }
    if (typeof name === 'string') {
      filters.name = name;
    }

    const sortStr = typeof sort === 'string' ? sort : undefined;
    const pageNum = typeof page === 'string' && !isNaN(Number(page)) ? Math.max(1, parseInt(page, 10)) : undefined;
    const limitNum = typeof limit === 'string' && !isNaN(Number(limit)) ? Math.max(1, parseInt(limit, 10)) : undefined;

    const result = await entityStorage.findAll({
      filters,
      sort: sortStr,
      page: pageNum,
      limit: limitNum
    });
    res.status(200).json(result);
  } catch (error) {
    next(error);
  }
});

router.get('/expensive', async (req, res, next) => {
  try {
    const items = await entityStorage.findExpensive();
    res.status(200).json(items);
```

```
  } catch (error) {
    next(error);
  }
});

router.get('/:id', async (req, res, next) => {
  try {
    const item = await entityStorage.findById(req.params.id as string);
    if (!item) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    res.status(200).json(item);
  } catch (error) {
    next(error);
  }
});

router.post('/', validate(createSchema), async (req, res, next) => {
  try {
    const newItem = await entityStorage.create(req.body);
    res.status(201).json(newItem);
  } catch (error) {
    next(error);
  }
});

router.put('/:id', validate(updateSchema), async (req, res, next) => {
  try {
    const updated = await entityStorage.update(req.params.id as string, req.body);
    if (!updated) {
      res.status(404).json({ error: 'Item not found' });
      return;
```

```
    }
    res.status(200).json(updated);
  } catch (error) {
    next(error);
  }
});

router.delete('/:id', async (req, res, next) => {
  try {
    const success = await entityStorage.delete(req.params.id as string);
    if (!success) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    res.status(204).end();
  } catch (error) {
    next(error);
  }
});

export default router;
```

src/storage/entity.ts

```
import { WishlistItem } from '../models/entity.model';
import { Entity, EntityFilters } from '../schemas/entity.schema';

export const entityStorage = {
  async findAll(options?: {
    filters?: EntityFilters;
    sort?: string;
    page?: number;
    limit?: number;
```

```
}): Promise<{
    data: any[];
    pagination: {
      page: number;
      limit: number;
      total: number;
      pages: number;
    };
}> {
    const filters = options?.filters;
    const sortStr = options?.sort;
    const page = options?.page ?? 1;
    const limit = options?.limit ?? 10;

    const query: any = {};
    if (filters) {
      if (filters.priority) {
        query.priority = filters.priority;
      }
      if (filters.maxPrice !== undefined) {
        query.price = { $lte: filters.maxPrice };
      }
      if (filters.name) {
        query.name = { $regex: filters.name, $options: 'i' };
      }
    }

    let sortObj: any = { createdAt: -1 };
    if (sortStr) {
      const isDesc = sortStr.startsWith('-');
      const field = isDesc ? sortStr.substring(1) : sortStr;
      const allowedFields = ['price', 'name', 'createdAt', 'priority'];
      if (allowedFields.includes(field)) {
```

```
            sortObj = { [field]: isDesc ? -1 : 1 };
        }
    }

    const skip = (page - 1) * limit;

    const [total, data] = await Promise.all([
        WishlistItem.countDocuments(query),
        WishlistItem.find(query)
            .sort(sortObj)
            .skip(skip)
            .limit(limit)
    ]);

    return {
        data,
        pagination: {
            page,
            limit,
            total,
            pages: Math.ceil(total / limit)
        }
    };
},

async findExpensive(): Promise<any[]> {
    return WishlistItem.find({ price: { $gt: 100 } });
},

async findById(id: string): Promise<any | null> {
    return WishlistItem.findById(id);
},
```

```
  async create(item: Omit<Entity, 'id' | 'createdAt' | 'updatedAt'>): Promise<any> {
    return WishlistItem.create(item);
  },

  async update(id: string, item: Partial<Omit<Entity, 'id' | 'createdAt' | 'updatedAt'>>):
Promise<any | null> {
    return WishlistItem.findByIdAndUpdate(id, item, {
      new: true,
      runValidators: true
    });
  },

  async delete(id: string): Promise<boolean> {
    const result = await WishlistItem.findByIdAndDelete(id);
    return result !== null;
  }
};
```

tests/entity.test.ts

```
import request from 'supertest';
import app from '../src/app';
import { entityStorage } from '../src/storage/entity';
import { WishlistItem } from '../src/models/entity.model';
import { connectDB } from '../src/config/database';
import mongoose from 'mongoose';
import dotenv from 'dotenv';

// Load environment variables for testing
dotenv.config();

beforeAll(async () => {
  await connectDB();
```

```
});

afterAll(async () => {
  await mongoose.connection.close();
});

describe('Wishlist API Integration Tests', () => {
  beforeEach(async () => {
    await WishlistItem.deleteMany({});
  });

  const validItem = {
    name: 'PlayStation 5',
    description: 'Next-gen gaming console',
    price: 499.99,
    priority: 'high' as const,
    url: 'https://sony.com/ps5'
  };

  it('GET /entities should return an empty array initially', async () => {
    const res = await request(app).get('/entities');
    expect(res.status).toBe(200);
    expect(res.body.data).toEqual([]);
    expect(res.body.pagination).toBeDefined();
    expect(res.body.pagination.total).toBe(0);
  });

  it('GET /entities/:id should return 400 for invalid ID format', async () => {
    const res = await request(app).get('/entities/non-existent-uuid');
    expect(res.status).toBe(400);
  });

  it('GET /entities/:id should return 404 for valid format but non-existent ID', async () => {
```

```
    const res = await request(app).get('/entities/507f1f77bcf86cd799439011');
    expect(res.status).toBe(404);
    expect(res.body).toEqual({ error: 'Item not found' });
});

it('POST /entities should create a wishlist item and return 201', async () => {
    const res = await request(app).post('/entities').send(validItem);
    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty('id');
    expect(res.body).toHaveProperty('createdAt');
    expect(res.body).toHaveProperty('updatedAt');
    expect(res.body.name).toBe(validItem.name);
    expect(res.body.description).toBe(validItem.description);
    expect(res.body.price).toBe(validItem.price);
    expect(res.body.priority).toBe(validItem.priority);
    expect(res.body.url).toBe(validItem.url);
});

it('POST /entities should return 400 if name is empty', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, name: '' });
    expect(res.status).toBe(400);
    expect(res.body.status).toBe('error');
    expect(res.body.message).toBe('Validation error');
});

it('POST /entities should return 400 if name exceeds 100 characters', async () => {
    const longName = 'a'.repeat(101);
    const res = await request(app).post('/entities').send({ ...validItem, name: longName });
    expect(res.status).toBe(400);
    expect(res.body.status).toBe('error');
});

it('POST /entities should return 400 if description exceeds 500 characters', async () => {
```

```
    const longDesc = 'a'.repeat(501);
    const res = await request(app).post('/entities').send({ ...validItem, description: longDesc });
    expect(res.status).toBe(400);
});

it('POST /entities should return 400 if price is negative', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, price: -10 });
    expect(res.status).toBe(400);
});

it('POST /entities should return 400 if priority is invalid', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, priority: 'critical' });
    expect(res.status).toBe(400);
});

it('POST /entities should return 400 if url is invalid URL format', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, url: 'not-a-valid-url' });
    expect(res.status).toBe(400);
});

it('GET /entities/:id should return the item if it exists', async () => {
    const created = await entityStorage.create(validItem);
    const res = await request(app).get(`/entities/${created.id}`);
    expect(res.status).toBe(200);
    expect(res.body.id).toBe(created.id);
    expect(res.body.name).toBe(validItem.name);
});

it('PUT /entities/:id should update wishlist item and return 200', async () => {
    const created = await entityStorage.create(validItem);
    const updateData = { name: 'PlayStation 5 Slim', price: 449.99 };
    const res = await request(app).put(`/entities/${created.id}`).send(updateData);
    expect(res.status).toBe(200);
```

```
    expect(res.body.id).toBe(created.id);
    expect(res.body.name).toBe(updateData.name);
    expect(res.body.price).toBe(updateData.price);
    expect(res.body.priority).toBe(validItem.priority);
  });

  it('PUT /entities/:id should return 400 if updating invalid ID format', async () => {
    const res = await request(app).put('/entities/non-existent-uuid').send({ name: 'New Name' });
    expect(res.status).toBe(400);
  });

  it('PUT /entities/:id should return 404 if updating valid format but non-existent ID', async () => {
    const res = await request(app).put('/entities/507f1f77bcf86cd799439011').send({ name: 'New Name' });
    expect(res.status).toBe(404);
  });

  it('DELETE /entities/:id should delete item and return 204', async () => {
    const created = await entityStorage.create(validItem);
    const delRes = await request(app).delete(`/entities/${created.id}`);
    expect(delRes.status).toBe(204);

    const getRes = await request(app).get(`/entities/${created.id}`);
    expect(getRes.status).toBe(404);
  });

  it('DELETE /entities/:id should return 400 if deleting invalid ID format', async () => {
    const res = await request(app).delete('/entities/non-existent-uuid');
    expect(res.status).toBe(400);
  });

  it('DELETE /entities/:id should return 404 if deleting valid format but non-existent ID', async () => {
```

```
    const res = await request(app).delete('/entities/507f1f77bcf86cd799439011');
    expect(res.status).toBe(404);
  });

  it('GET /entities?priority=high should return only high-priority items', async () => {
    await entityStorage.create({ ...validItem, name: 'Item 1', priority: 'high' });
    await entityStorage.create({ ...validItem, name: 'Item 2', priority: 'low' });
    await entityStorage.create({ ...validItem, name: 'Item 3', priority: 'high' });

    const res = await request(app).get('/entities?priority=high');
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBe(2);
    expect(res.body.data.every((item: any) => item.priority === 'high')).toBe(true);
  });

  it('GET /entities?maxPrice=100 should return only items with price <= 100', async () => {
    await entityStorage.create({ ...validItem, name: 'Cheap 1', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Expensive', price: 150 });
    await entityStorage.create({ ...validItem, name: 'Cheap 2', price: 100 });

    const res = await request(app).get('/entities?maxPrice=100');
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBe(2);
    expect(res.body.data.every((item: any) => item.price <= 100)).toBe(true);
  });

  it('GET /entities?name=Play should return only items with matching names (case-
insensitive)', async () => {
    await entityStorage.create({ ...validItem, name: 'PlayStation 5' });
    await entityStorage.create({ ...validItem, name: 'Xbox Series X' });
    await entityStorage.create({ ...validItem, name: 'Nintendo Switch' });

    const res = await request(app).get('/entities?name=play');
    expect(res.status).toBe(200);
```

```
    expect(res.body.data.length).toBe(1);
    expect(res.body.data[0].name).toBe('PlayStation 5');
});

it('GET /entities with multiple filters should return only matching items', async () => {
    await entityStorage.create({ ...validItem, name: 'Low Cheap', priority: 'low', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Low Expensive', priority: 'low', price: 150 });
    await entityStorage.create({ ...validItem, name: 'High Cheap', priority: 'high', price: 50 });

    const res = await request(app).get('/entities?priority=low&maxPrice=100');
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBe(1);
    expect(res.body.data[0].name).toBe('Low Cheap');
});

it('GET /entities pagination should return paginated list of items', async () => {
    for (let i = 1; i <= 15; i++) {
        await entityStorage.create({ ...validItem, name: `Item ${i}`, price: i * 10 });
    }

    const res = await request(app).get('/entities?page=2&limit=5');
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBe(5);
    expect(res.body.pagination.page).toBe(2);
    expect(res.body.pagination.limit).toBe(5);
    expect(res.body.pagination.total).toBe(15);
    expect(res.body.pagination.pages).toBe(3);
});

it('GET /entities sort should return sorted list of items', async () => {
    await entityStorage.create({ ...validItem, name: 'Item A', price: 300 });
    await entityStorage.create({ ...validItem, name: 'Item B', price: 100 });
    await entityStorage.create({ ...validItem, name: 'Item C', price: 200 });
```

```
    // Sort ascending by price
    const resAsc = await request(app).get('/entities?sort=price');
    expect(resAsc.status).toBe(200);
    expect(resAsc.body.data[0].price).toBe(100);
    expect(resAsc.body.data[1].price).toBe(200);
    expect(resAsc.body.data[2].price).toBe(300);

    // Sort descending by price
    const resDesc = await request(app).get('/entities?sort=-price');
    expect(resDesc.status).toBe(200);
    expect(resDesc.body.data[0].price).toBe(300);
    expect(resDesc.body.data[1].price).toBe(200);
    expect(resDesc.body.data[2].price).toBe(100);
  });

  it('GET /entities/expensive should return items with price > 100', async () => {
    await entityStorage.create({ ...validItem, name: 'Cheap', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Threshold', price: 100 });
    await entityStorage.create({ ...validItem, name: 'Expensive 1', price: 150 });
    await entityStorage.create({ ...validItem, name: 'Expensive 2', price: 300 });

    const res = await request(app).get('/entities/expensive');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(2);
    expect(res.body.some((item: any) => item.name === 'Expensive 1')).toBe(true);
    expect(res.body.some((item: any) => item.name === 'Expensive 2')).toBe(true);
  });
});
```

3.7. Завдання 7: Тестування з mongodb-memory-server

package.json

```
{
 "name": "nodejs",
 "version": "1.0.0",
 "description": "",
 "main": "index.js",
 "scripts": {
  "dev": "ts-node src/server.ts",
  "build": "tsc",
  "test": "jest",
  "test:coverage": "jest --coverage"
 },
 "repository": {
  "type": "git",
  "url": "git+https://github.com/IhnatiukAndrii/nodejs-labs.git"
 },
 "keywords": [],
 "author": "",
 "license": "ISC",
 "type": "commonjs",
 "bugs": {
  "url": "https://github.com/IhnatiukAndrii/nodejs-labs/issues"
 },
 "homepage": "https://github.com/IhnatiukAndrii/nodejs-labs#readme",
 "dependencies": {
  "cors": "^2.8.6",
  "dotenv": "^17.4.2",
  "express": "^5.2.1",
  "mongoose": "^9.6.3",
  "zod": "^4.4.3"
 },
 "devDependencies": {
```

```
    "@types/cors": "^2.8.19",
    "@types/express": "^5.0.6",
    "@types/jest": "^30.0.0",
    "@types/supertest": "^7.2.0",
    "jest": "^30.4.2",
    "mongodb-memory-server": "^11.2.0",
    "supertest": "^7.2.2",
    "ts-jest": "^29.4.11",
    "ts-node": "^10.9.2",
    "typescript": "^6.0.3"
  }
}
```

tests/setup.ts

```
import { MongoMemoryServer } from 'mongodb-memory-server';
import mongoose from 'mongoose';

let mongod: MongoMemoryServer;

export const connect = async (): Promise<void> => {
    mongod = await MongoMemoryServer.create();
    const uri = mongod.getUri();
    await mongoose.connect(uri);
};

export const closeDatabase = async (): Promise<void> => {
    if (mongoose.connection.readyState !== 0) {
        await mongoose.connection.dropDatabase();
        await mongoose.connection.close();
    }
    if (mongod) {
        await mongod.stop();
```

```
  }
};

export const clearDatabase = async (): Promise<void> => {
  const collections = mongoose.connection.collections;
  for (const key in collections) {
    const collection = collections[key];
    await collection.deleteMany({});
  }
};
```

tests/entity.test.ts

```
import request from 'supertest';
import app from '../src/app';
import { entityStorage } from '../src/storage/entity';
import { WishlistItem } from '../src/models/entity.model';
import { connect, closeDatabase, clearDatabase } from './setup';
import { errorHandler } from '../src/middleware';

beforeAll(async () => {
  await connect();
});

afterAll(async () => {
  await closeDatabase();
});

afterEach(async () => {
  await clearDatabase();
});

describe('Wishlist API Integration Tests', () => {
```

```
const validItem = {
    name: 'PlayStation 5',
    description: 'Next-gen gaming console',
    price: 499.99,
    priority: 'high' as const,
    url: 'https://sony.com/ps5'
};

it('GET /entities should return an empty array initially', async () => {
    const res = await request(app).get('/entities');
    expect(res.status).toBe(200);
    expect(res.body.data).toEqual([]);
    expect(res.body.pagination).toBeDefined();
    expect(res.body.pagination.total).toBe(0);
});

it('GET /entities/:id should return 400 for invalid ID format', async () => {
    const res = await request(app).get('/entities/non-existent-uuid');
    expect(res.status).toBe(400);
});

it('GET /entities/:id should return 404 for valid format but non-existent ID', async () => {
    const res = await request(app).get('/entities/507f1f77bcf86cd799439011');
    expect(res.status).toBe(404);
    expect(res.body).toEqual({ error: 'Item not found' });
});

it('POST /entities should create a wishlist item and return 201', async () => {
    const res = await request(app).post('/entities').send(validItem);
    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty('id');
    expect(res.body).toHaveProperty('createdAt');
```

```
    expect(res.body).toHaveProperty('updatedAt');
    expect(res.body.name).toBe(validItem.name);
    expect(res.body.description).toBe(validItem.description);
    expect(res.body.price).toBe(validItem.price);
    expect(res.body.priority).toBe(validItem.priority);
    expect(res.body.url).toBe(validItem.url);
});

it('POST /entities should return 400 if name is empty', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, name: '' });
    expect(res.status).toBe(400);
    expect(res.body.status).toBe('error');
    expect(res.body.message).toBe('Validation error');
});

it('POST /entities should return 400 if name exceeds 100 characters', async () => {
    const longName = 'a'.repeat(101);
    const res = await request(app).post('/entities').send({ ...validItem, name: longName });
    expect(res.status).toBe(400);
    expect(res.body.status).toBe('error');
});

it('POST /entities should return 400 if description exceeds 500 characters', async () => {
    const longDesc = 'a'.repeat(501);
    const res = await request(app).post('/entities').send({ ...validItem, description: longDesc });
    expect(res.status).toBe(400);
});

it('POST /entities should return 400 if price is negative', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, price: -10 });
    expect(res.status).toBe(400);
});
```

```
  it('POST /entities should return 400 if priority is invalid', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, priority: 'critical' });
    expect(res.status).toBe(400);
  });

  it('POST /entities should return 400 if url is invalid URL format', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, url: 'not-a-valid-url' });
    expect(res.status).toBe(400);
  });

  it('GET /entities/:id should return the item if it exists', async () => {
    const created = await entityStorage.create(validItem);
    const res = await request(app).get(`/entities/${created.id}`);
    expect(res.status).toBe(200);
    expect(res.body.id).toBe(created.id);
    expect(res.body.name).toBe(validItem.name);
  });

  it('PUT /entities/:id should update wishlist item and return 200', async () => {
    const created = await entityStorage.create(validItem);
    const updateData = { name: 'PlayStation 5 Slim', price: 449.99 };
    const res = await request(app).put(`/entities/${created.id}`).send(updateData);
    expect(res.status).toBe(200);
    expect(res.body.id).toBe(created.id);
    expect(res.body.name).toBe(updateData.name);
    expect(res.body.price).toBe(updateData.price);
    expect(res.body.priority).toBe(validItem.priority);
  });

  it('PUT /entities/:id should return 400 if updating invalid ID format', async () => {
    const res = await request(app).put('/entities/non-existent-uuid').send({ name: 'New
Name' });
    expect(res.status).toBe(400);
  });
```

```
  it('PUT /entities/:id should return 404 if updating valid format but non-existent ID', async () =>
{
    const res = await request(app).put('/entities/507f1f77bcf86cd799439011').send({ name:
'New Name' });
    expect(res.status).toBe(404);
  });

  it('DELETE /entities/:id should delete item and return 204', async () => {
    const created = await entityStorage.create(validItem);
    const delRes = await request(app).delete(`/entities/${created.id}`);
    expect(delRes.status).toBe(204);

    const getRes = await request(app).get(`/entities/${created.id}`);
    expect(getRes.status).toBe(404);
  });

  it('DELETE /entities/:id should return 400 if deleting invalid ID format', async () => {
    const res = await request(app).delete('/entities/non-existent-uuid');
    expect(res.status).toBe(400);
  });

  it('DELETE /entities/:id should return 404 if deleting valid format but non-existent ID', async ()
=> {
    const res = await request(app).delete('/entities/507f1f77bcf86cd799439011');
    expect(res.status).toBe(404);
  });

  it('GET /entities?priority=high should return only high-priority items', async () => {
    await entityStorage.create({ ...validItem, name: 'Item 1', priority: 'high' });
    await entityStorage.create({ ...validItem, name: 'Item 2', priority: 'low' });
    await entityStorage.create({ ...validItem, name: 'Item 3', priority: 'high' });

    const res = await request(app).get('/entities?priority=high');
```

```
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBe(2);
    expect(res.body.data.every((item: any) => item.priority === 'high')).toBe(true);
  });

  it('GET /entities?maxPrice=100 should return only items with price <= 100', async () => {
    await entityStorage.create({ ...validItem, name: 'Cheap 1', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Expensive', price: 150 });
    await entityStorage.create({ ...validItem, name: 'Cheap 2', price: 100 });

    const res = await request(app).get('/entities?maxPrice=100');
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBe(2);
    expect(res.body.data.every((item: any) => item.price <= 100)).toBe(true);
  });

  it('GET /entities?name=Play should return only items with matching names (case-
insensitive)', async () => {
    await entityStorage.create({ ...validItem, name: 'PlayStation 5' });
    await entityStorage.create({ ...validItem, name: 'Xbox Series X' });
    await entityStorage.create({ ...validItem, name: 'Nintendo Switch' });

    const res = await request(app).get('/entities?name=play');
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBe(1);
    expect(res.body.data[0].name).toBe('PlayStation 5');
  });

  it('GET /entities with multiple filters should return only matching items', async () => {
    await entityStorage.create({ ...validItem, name: 'Low Cheap', priority: 'low', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Low Expensive', priority: 'low', price: 150 });
    await entityStorage.create({ ...validItem, name: 'High Cheap', priority: 'high', price: 50 });

    const res = await request(app).get('/entities?priority=low&maxPrice=100');
```

```
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBe(1);
    expect(res.body.data[0].name).toBe('Low Cheap');
});

it('GET /entities pagination should return paginated list of items', async () => {
    for (let i = 1; i <= 15; i++) {
        await entityStorage.create({ ...validItem, name: `Item ${i}`, price: i * 10 });
    }

    const res = await request(app).get('/entities?page=2&limit=5');
    expect(res.status).toBe(200);
    expect(res.body.data.length).toBe(5);
    expect(res.body.pagination.page).toBe(2);
    expect(res.body.pagination.limit).toBe(5);
    expect(res.body.pagination.total).toBe(15);
    expect(res.body.pagination.pages).toBe(3);
});

it('GET /entities sort should return sorted list of items', async () => {
    await entityStorage.create({ ...validItem, name: 'Item A', price: 300 });
    await entityStorage.create({ ...validItem, name: 'Item B', price: 100 });
    await entityStorage.create({ ...validItem, name: 'Item C', price: 200 });

    // Sort ascending by price
    const resAsc = await request(app).get('/entities?sort=price');
    expect(resAsc.status).toBe(200);
    expect(resAsc.body.data[0].price).toBe(100);
    expect(resAsc.body.data[1].price).toBe(200);
    expect(resAsc.body.data[2].price).toBe(300);

    // Sort descending by price
    const resDesc = await request(app).get('/entities?sort=-price');
```

```
    expect(resDesc.status).toBe(200);
    expect(resDesc.body.data[0].price).toBe(300);
    expect(resDesc.body.data[1].price).toBe(200);
    expect(resDesc.body.data[2].price).toBe(100);
  });

  it('GET /entities/expensive should return items with price > 100', async () => {
    await entityStorage.create({ ...validItem, name: 'Cheap', price: 50 });
    await entityStorage.create({ ...validItem, name: 'Threshold', price: 100 });
    await entityStorage.create({ ...validItem, name: 'Expensive 1', price: 150 });
    await entityStorage.create({ ...validItem, name: 'Expensive 2', price: 300 });

    const res = await request(app).get('/entities/expensive');
    expect(res.status).toBe(200);
    expect(res.body.length).toBe(2);
    expect(res.body.some((item: any) => item.name === 'Expensive 1')).toBe(true);
    expect(res.body.some((item: any) => item.name === 'Expensive 2')).toBe(true);
  });
});

describe('WishlistItem Model Unit Tests', () => {
  const validData = {
    name: 'Nintendo Switch',
    description: 'Hybrid console',
    price: 299.99,
    priority: 'medium' as const,
    url: 'https://nintendo.com/switch'
  };

  it('should create a valid model with all fields', async () => {
    const item = new WishlistItem(validData);
    await item.validate();
    expect(item.name).toBe(validData.name);
```

```
    expect(item.description).toBe(validData.description);
    expect(item.price).toBe(validData.price);
    expect(item.priority).toBe(validData.priority);
    expect(item.url).toBe(validData.url);
});

it('should set default priority to medium if not provided', async () => {
    const item = new WishlistItem({
        name: 'Book',
        price: 15.99
    });
    expect(item.priority).toBe('medium');
});

it('should fail validation if name is empty', async () => {
    const item = new WishlistItem({ ...validData, name: '' });
    await expect(item.validate()).rejects.toThrow();
});

it('should fail validation if name is longer than 100 characters', async () => {
    const item = new WishlistItem({ ...validData, name: 'a'.repeat(101) });
    await expect(item.validate()).rejects.toThrow();
});

it('should fail validation if description is longer than 500 characters', async () => {
    const item = new WishlistItem({ ...validData, description: 'a'.repeat(501) });
    await expect(item.validate()).rejects.toThrow();
});

it('should fail validation if price is less than 0.01', async () => {
    const item = new WishlistItem({ ...validData, price: 0 });
    await expect(item.validate()).rejects.toThrow();
});
```

```
it('should fail validation if priority is invalid', async () => {
    const item = new WishlistItem({ ...validData, priority: 'critical' as any });
    await expect(item.validate()).rejects.toThrow();
});

it('should fail validation if URL format is invalid', async () => {
    const item = new WishlistItem({ ...validData, url: 'invalid-url' });
    await expect(item.validate()).rejects.toThrow();
});

it('should successfully validate empty URL (optional field)', async () => {
    const item = new WishlistItem({
        name: 'Self-improvement book',
        price: 19.99
    });
    await expect(item.validate()).resolves.not.toThrow();
});

it('should check virtual isExpensive property (true if price > 100)', async () => {
    const expensiveItem = new WishlistItem({ ...validData, price: 100.01 }) as any;
    expect(expensiveItem.isExpensive).toBe(true);

    const cheapItem = new WishlistItem({ ...validData, price: 100.00 }) as any;
    expect(cheapItem.isExpensive).toBe(false);
});

it('should automatically set createdAt and updatedAt timestamps upon save', async () => {
    const item = new WishlistItem(validData);
    const saved = await item.save() as any;
    expect(saved.createdAt).toBeDefined();
    expect(saved.updatedAt).toBeDefined();
    expect(saved.createdAt).toBeInstanceOf(Date);
```

```
    expect(saved.updatedAt).toBeInstanceOf(Date);
  });

  it('should validate empty/blank URL properly', async () => {
    const item = new WishlistItem({
      name: 'Item with empty string url',
      price: 15.00,
      url: ''
    });
    await expect(item.validate()).resolves.not.toThrow();
  });
});

describe('errorHandler middleware unit tests', () => {
  let mockRequest: any;
  let mockResponse: any;
  let mockNext: any;

  beforeEach(() => {
    mockRequest = {};
    mockResponse = {
      status: jest.fn().mockReturnThis(),
      json: jest.fn().mockReturnThis()
    };
    mockNext = jest.fn();
  });

  it('should handle Mongoose ValidationError', () => {
    const err = {
      name: 'ValidationError',
      errors: {
        name: { path: 'name', message: 'Name is required' }
      }
```

```
    };
    errorHandler(err, mockRequest, mockResponse, mockNext);
    expect(mockResponse.status).toHaveBeenCalledWith(400);
    expect(mockResponse.json).toHaveBeenCalledWith({
      status: 'error',
      message: 'Validation error',
      errors: [{ path: 'name', message: 'Name is required' }]
    });
});

it('should handle duplicate key error (11000)', () => {
    const err = {
      code: 11000,
      keyValue: { name: 'PlayStation 5' }
    };
    errorHandler(err, mockRequest, mockResponse, mockNext);
    expect(mockResponse.status).toHaveBeenCalledWith(400);
    expect(mockResponse.json).toHaveBeenCalledWith({
      status: 'error',
      message: 'Duplicate key error',
      keyValue: { name: 'PlayStation 5' }
    });
});

it('should handle fallback 500 error', () => {
    const err = new Error('Test unhandled error');
    errorHandler(err, mockRequest, mockResponse, mockNext);
    expect(mockResponse.status).toHaveBeenCalledWith(500);
    expect(mockResponse.json).toHaveBeenCalledWith({
      status: 'error',
      message: 'Test unhandled error'
    });
});
```

```
});

describe('Route catch blocks', () => {
  it('GET /entities should handle errors and call next', async () => {
    const spy = jest.spyOn(entityStorage, 'findAll').mockRejectedValue(new Error('DB Error'));
    const res = await request(app).get('/entities');
    expect(res.status).toBe(500);
    expect(res.body.message).toBe('DB Error');
    spy.mockRestore();
  });

  it('GET /entities/expensive should handle errors and call next', async () => {
    const spy = jest.spyOn(entityStorage, 'findExpensive').mockRejectedValue(new Error('DB
Error'));
    const res = await request(app).get('/entities/expensive');
    expect(res.status).toBe(500);
    expect(res.body.message).toBe('DB Error');
    spy.mockRestore();
  });

  it('GET /entities/:id should handle errors and call next', async () => {
    const spy = jest.spyOn(entityStorage, 'findById').mockRejectedValue(new Error('DB Error'));
    const res = await request(app).get('/entities/507f1f77bcf86cd799439011');
    expect(res.status).toBe(500);
    expect(res.body.message).toBe('DB Error');
    spy.mockRestore();
  });

  it('POST /entities should handle errors and call next', async () => {
    const spy = jest.spyOn(entityStorage, 'create').mockRejectedValue(new Error('DB Error'));
    const res = await request(app).post('/entities').send({
      name: 'Test item',
      price: 10,
      priority: 'medium'
```

```
    });
    expect(res.status).toBe(500);
    spy.mockRestore();
  });

  it('PUT /entities/:id should handle errors and call next', async () => {
    const spy = jest.spyOn(entityStorage, 'update').mockRejectedValue(new Error('DB Error'));
    const res = await request(app).put('/entities/507f1f77bcf86cd799439011').send({ name:
'New Name' });
    expect(res.status).toBe(500);
    spy.mockRestore();
  });

  it('DELETE /entities/:id should handle errors and call next', async () => {
    const spy = jest.spyOn(entityStorage, 'delete').mockRejectedValue(new Error('DB Error'));
    const res = await request(app).delete('/entities/507f1f77bcf86cd799439011');
    expect(res.status).toBe(500);
    spy.mockRestore();
  });
});

describe('Storage edge cases', () => {
  it('findAll should work without any options', async () => {
    const result = await entityStorage.findAll();
    expect(result.data).toEqual([]);
  });

  it('findAll with invalid sort field should use default sort', async () => {
    await entityStorage.create({
      name: 'Item A',
      price: 10,
      priority: 'medium'
    });
    const result = await entityStorage.findAll({ sort: 'invalidField' });
```

```
    expect(result.data.length).toBe(1);
  });
});
```

Тестування

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	93.93	100	100	
src	100	100	100	100	
app.ts	100	100	100	100	
src/middleware	100	85.71	100	100	
errorHandler.ts	100	85.71	100	100	32,51
index.ts	100	100	100	100	
validation.ts	100	100	100	100	
src/models	100	100	100	100	
entity.model.ts	100	100	100	100	
src/routes	100	100	100	100	
entity.ts	100	100	100	100	
src/schemas	100	100	100	100	
entity.schema.ts	100	100	100	100	
src/storage	100	100	100	100	
entity.ts	100	100	100	100	
tests	100	50	100	100	
setup.ts	100	50	100	100	13-17

```
Test Suites: 1 passed, 1 total
Tests:       47 passed, 47 total
Snapshots:   0 total
Time:        4.459 s, estimated 8 s
```

Висновок: в ході виконання роботи набув практичних навичок роботи з MongoDB та Mongoose шляхом міграції REST API з лабораторної роботи №3 з in-memory сховища на повноцінну базу даних, зокрема: • налаштування MongoDB Atlas та MongoDB Compass для хмарного зберігання та візуального управління даними; • підключення Mongoose до Express-застосунку з паттерном graceful shutdown; • проєктування Mongoose-схеми з валідацією, дефолтами та віртуальними вла- стивостями; • міграція CRUD-операцій з Map-сховища на Mongoose Model; • централізована обробка специфічних помилок MongoDB (CastError, ValidationError, duplicate key); • реалізація фільтрації, сортування та пагінації через Mongoose Query API; • тестування API з використанням mongodb-memory-server замість реальної бази даних.